

Εισαγωγή Αναφοράς:

Σκοπός της άσκησης ήταν η δημιουργία και η σύγκριση 2 δομών δεδομένων (Binary Search Tree και Binary Heap) για την εύρεση του πιο αποδοτικού τρόπου εισαγωγής και διαγραφής στοιχείων. Η υλοποίηση του BST έγινε με δυναμική παραχώρηση μνήμης και με ένα array Nx3 ακεραίων, ενώ η υλοποίηση του Heap έγινε μόνο με χρήση array.

1)BST με χρήση array:

Το array διαστάσεων Nx3 περιλάμβανε $N = 10^6$ κόμβους του Δυαδικού Δένδρου Αναζήτησης για καθέναν από τους οποίους χρειάστηκε να αποθηκεύσουμε το κλειδί του (ακέραιος) και τις θέσεις του αριστερού και του δεξιού του παιδιού στο array. Πριν την εισαγωγή των στοιχείων, το array αρχικοποιήθηκε στο -1.

Η εισαγωγή των κλειδιών έγινε με αναδρομή, ξεκινώντας κάθε φορά από την ρίζα του εκάστοτε υποδένδρου και συγκρίνοντας τα παιδιά του, μέχρι να βρεθεί κενή θέση. Όταν γίνει αυτό, αποθηκεύουμε το στοιχείο μας και ενημερώνουμε την μεταβλητή avail.

Για την αναζήτηση ενός στοιχείου, διασχίζουμε το array με αντίστοιχο τρόπο και επιστρέφουμε την τιμή του για επιτυχημένη αναζήτηση και -1 για αποτυχημένη αναζήτηση.

Για την διαγραφή ακολουθείται η ίδια διαδικασία με την αναζήτηση, μόνο που όταν βρεθεί το επιθυμητό στοιχείο προχωράμε στις εξής υποπεριπτώσεις: Αν έχει ένα ή κανένα παιδί, αντικαθιστούμε με το παιδί του και τον διαγράφουμε. Αν έχει 2 παιδιά, αντικαθιστούμε με την ελάχιστη τιμή του δεξιού του υποδένδρου και τον διαγράφουμε. Σε κάθε περίπτωση ενημερώνουμε το avail.

2)BST με χρήση δυναμικής παραχώρησης μνήμης:

Χρησιμοποιήθηκε το χρήσιμο κομμάτι του κώδικα που μας δόθηκε στο φροντιστήριο, ο οποίος ακολουθεί την ίδια λογική με παραπάνω. Αυτή τη φορά οι κόμβοι του δένδρου δημιουργήθηκαν δυναμικά, με χρήση της κλάσης BinNode.

3)Heap με χρήση array:

Και πάλι χρησιμοποιήθηκε ο κώδικας του φροντιστηρίου, κατά τον οποίο γίνονται τα εξής:

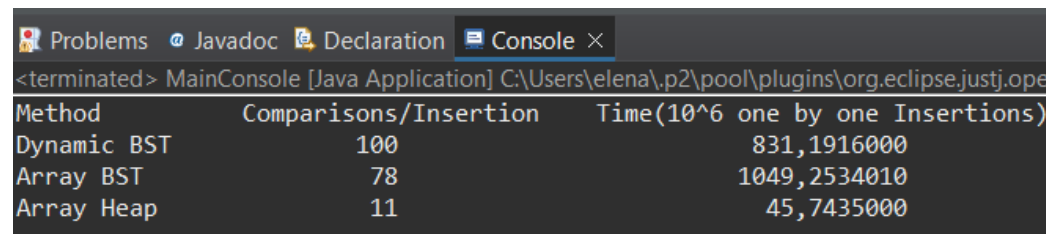
Η εισαγωγή κλειδιού γίνεται στην επόμενη κενή θέση του Heap, δηλαδή στην $n++$ αν n είναι το πλήθος των στοιχείων που έχουν εισαχθεί. Κάθε φορά που δεν βρισκόμαστε στη ρίζα και που ο current κόμβος είναι μεγαλύτερος του πατέρα, κάνουμε swap τους 2 κόμβους και θέτουμε τον current κόμβο με τον πατέρα του.

Για την διαγραφή του μέγιστου κλειδιού κάνουμε swap τον τελευταίο κόμβο με την ελάχιστη τιμή ελαττώνοντας και το πλήθος των στοιχείων. Έπειτα, με την siftdown μετακινούμε τον κόμβο στην σωστή θέση, ώστε να μην παραβιάζεται ο κανόνας του complete BST.

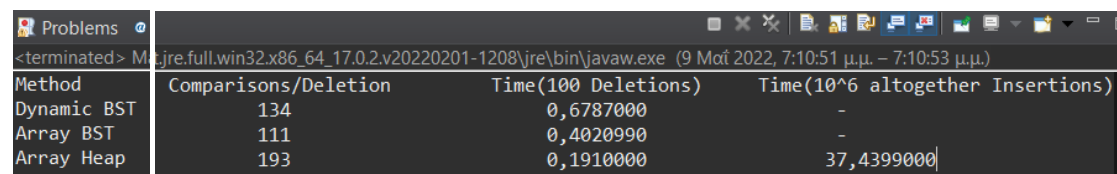
Για να εισάγουμε τα στοιχεία μονομιάς στο heap, αντί να κάνουμε πολλαπλά insert, καλούμε την buildHeap, η οποία κάνει την αρχικοποίηση με τα κατάλληλα siftdown.

4) Σύγκριση μεθόδων και τεκμηρίωση:

Κατά την εισαγωγή και την διαγραφή στοιχείων λάβαμε υπόψιν όλες τις συγκρίσεις και τις αναθέσεις που απαιτήθηκαν στην κάθε δομή. Όσο αναφορά στον χρόνο, χρησιμοποιήθηκε η συνάρτηση System.nanoTime() και τα αποτελέσματα εκτυπώθηκαν σε ms. Για το BST έγιναν οι εισαγωγές των 10^6 κλειδιών που μας δόθηκαν και μπήκαν σε τυχαία σειρά μέσω της Random() και αντίστοιχα οι διαγραφές των 100 κλειδιών που μας δόθηκαν, ενώ για το Heap έγιναν οι ίδιες εισαγωγές είτε μία μία, είτε όλες μαζί και οι διαγραφές των 100 μέγιστων κλειδιών. Παρακάτω φαίνονται όλες οι ζητούμενες μετρήσεις:



Method	Comparisons/Insertion	Time(10^6 one by one Insertions)
Dynamic BST	100	831,1916000
Array BST	78	1049,2534010
Array Heap	11	45,7435000



Method	Comparisons/Deletion	Time(100 Deletions)	Time(10^6 altogether Insertions)
Dynamic BST	134	0,6787000	-
Array BST	111	0,4020990	-
Array Heap	193	0,1910000	37,4399000

Η πολυπλοκότητα όσο αναφορά τον μέσο αριθμό συγκρίσεων ανά εισαγωγή και ανά διαγραφή είναι $O(\log n)$ για όλες τις μεθόδους, όπως επιβεβαιώνουν και τα αποτελέσματα με μικρές αποκλίσεις λόγω της συμπερίληψης των αναθέσεων που αυξάνουν γραμμικά την τελική μέτρηση.

Συγκρίνοντας τις 2 μεθόδους του BST παρατηρούμε ότι το δένδρο με array είναι αποδοτικότερο τόσο στην εισαγωγή όσο και στην διαγραφή ως προς τον αριθμό των συγκρίσεων. Ωστόσο, χρονικά βλέπουμε ότι το δυναμικό BST είναι πιο γρήγορο για

εισαγωγή (περίπου κατά 26%), ενώ το BST με array είναι πιο γρήγορο για διαγραφή (περίπου κατά 69%).

Τέλος, συγκρίνοντας τα παραπάνω με την μέθοδο του Heap, παρατηρούμε ότι αυτή η μέθοδος είναι κατά βάση αποδοτικότερη από το BST, εκτός από τον μέσο αριθμό συγκρίσεων ανά διαγραφή, ο οποίος είναι μεγαλύτερος.

Όσο αναφορά τους 2 τρόπους εισαγωγής, η εισαγωγή όλων των στοιχείων απευθείας στο heap είναι ταχύτερη από την εισαγωγή των στοιχείων ένα-ένα.

Πηγές:

- Φροντιστήριο μαθήματος
- Ιστοσελίδες όπως οι [oracle](#), [stackoverflow](#) και [delftstack](#)